



HUGO PAVY - MINES PARIS-PSL

Rapport de stage :
Machine learning pour la physique, application à
l'équation de Navier-Stokes

MARS 2025

Mon stage a été encadré par Fikri Hafid, directeur de recherche à RTE et professeur associé à l'ENS Paris Saclay.

Je remercie beaucoup Fikri de m'avoir permis de faire ces 6 mois chez RTE. Ce stage m'a appris beaucoup aussi bien d'un point de vue technique que sur le fonctionnement de la recherche ou sur le travail en entreprise. Merci aussi à John Redford de Mews Lab avec qui j'ai beaucoup travaillé et découvert sur les simulations numériques. Merci enfin à toutes les personnes qui m'ont aidé à avancer au sein de RTE ou de Mews Lab.

Mon mail : hugo.pavy@etu.minesparis.psl.eu

Mon linkedin : [Hugo Pavy](#)

Contact tuteur RTE : fikri.hafid@rte-france.com

Contact John Redford (notamment pour les données d'entraînement) : john.redford@mews-labs.com

Vous trouverez les différents codes utilisés sur cette page github : [lien du github](#)

Table des matières

1	Introduction	3
1.1	Contexte du stage	3
1.2	Contributions	3
2	Utilisation des Physics Informed Neural Networks (PINNs) pour la reconstruction d'écoulements	4
2.1	Principe des PINNs	4
2.2	Présentation du problème	5
2.3	Prise en compte des forces inertielles	5
2.4	Utilisation de la normalisation Z	6
2.5	Stratégie de mooving weights	6
2.6	Résultats et comparaison avec l'apprentissage supervisé classique	7
3	Utilisation des Physics Informed Kolmogorov-Arnold Networks (PIKANs)	9
3.1	Principe	9
3.2	Résultats et discussion	11
4	Prédictions d'écoulements avec les PINNs	13
4.1	Présentation du problème	13
4.2	Résultats	14
5	Deep Operator Network (Deep O Net) pour la prédiction d'écoulements	18
5.1	Principes	18
5.2	Résultats et discussion	20
6	Utilisation des Graph Neural Network (GNN)	21
6.1	Principe	21
6.2	Résultats et discussion	23
7	Conclusion	24
7.1	Bilan	24
7.2	Discussion et perspectives	24
A	Utilisation d'Onyxia pour accéder à des GPU	27
B	Apprendre le ML avec internet	28

1 Introduction

1.1 Contexte du stage

Ce document est le compte rendu d'un travail de 6 mois réalisé dans le cadre de mon premier stage de césure avant ma dernière année du cycle ingénieur civil des Mines de Paris. Il a été réalisé à RTE dans le pôle de gestion des actifs de la R&D.

Les sujets d'études du pôle gestion des actifs sont principalement de prédire des défaillances d'actifs (câbles, pylônes...), d'optimiser leur remplacement, et de concevoir des méthodes pour détecter les défaillances.

Le pôle de gestion des actifs développe un modèle qui cherche à prédire la fatigue des câbles. Dans ce modèle il est question de réaliser un grand nombre de fois des simulations d'écoulements autour d'un cylindre en 2D. Au vu du nombre très important de simulations, il serait avantageux de pouvoir accélérer grandement la vitesse pour simuler un écoulement. C'est là qu'intervient ce sujet de stage : on peut chercher à utiliser l'apprentissage profond afin de produire des simulations presque instantanément.

Durant ce stage j'ai travaillé sur les méthodes classiques de deep learning (DL) utilisées dans le contexte des Physics Informed Neuron Networks (PINNs). J'ai essayé et appris beaucoup de différentes architectures. Vous trouverez en annexe le meilleur contenu que j'ai trouvé sur internet pour me former sur les différentes notions.

Ce stage est la suite d'un premier stage de 3 mois à RTE réalisé par Issame Maghraoui, élève à Polytechnique. Je me suis appuyé sur ses travaux afin de réaliser mon travail.

1.2 Contributions

Durant ce stage je me suis donc intéressé à la technologie des PINNs. Dans un premier temps l'objectif a été de reconstruire des écoulements à partir de données partielles, de faire une sorte d'interpolation. Ensuite j'ai essayé de prédire des écoulements de câbles en mouvement. J'ai aussi exploré plusieurs architectures : Multi Layers Perceptron (MLP) classiques, Physics Informed Kolmogorov-Arnold Networks (PIKANs), Deep Neural Operator et Graph Neural Network (GNN).

J'ai eu la chance durant ce stage de présenter mon travail à la conférence DTE AICOMAS 2025. C'était une expérience très enrichissante pour moi autant de montrer mon travail que de voir celui des autres et le fonctionnement de la recherche. Merci à Fikri, John et Eric pour cette opportunité.

2 Utilisation des Physics Informed Neural Networks (PINNs) pour la reconstruction d'écoulements

Le code utilisé dans cette partie est trouvable sur ce lien : [lien du github](#)

Dans cette partie nous allons faire une première implémentation des PINNs. L'objectif va être d'à partir un certain nombre de données d'un écoulement de reconstruire l'écoulement dans sa totalité.

2.1 Principe des PINNs

Le papier qui présente la technologie des PINNs est celui ci : [Lu et al. \[2021b\]](#). Je vais rappeler rapidement le principe des PINNs mais il ne faut pas hésiter à revenir au papier initial.

On considère un écoulement que l'on regarde de façon Eulérienne avec la vitesse en x $u(x, y, t)$ la vitesse en y $v(x, y, t)$ et le champ de pression $p(x, y, t)$.

On entraîne alors un Multi Layer Perceptron (MLP) de cette forme :

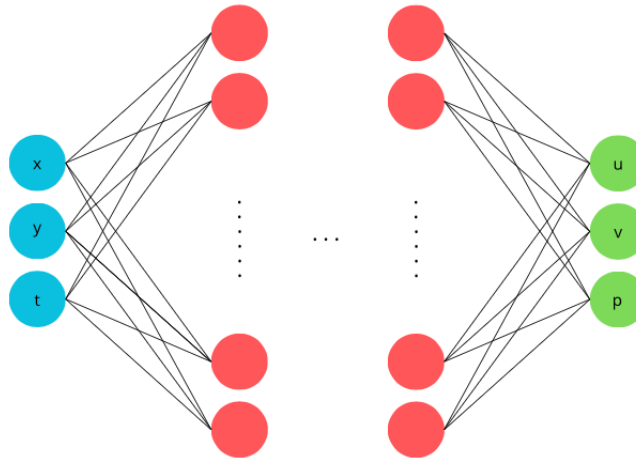


FIGURE 1 – MLP utilisé

Il va nous fournir des champs $\hat{u}(x, y, t)$, $\hat{v}(x, y, t)$, $\hat{p}(x, y, t)$ qui vont être optimisés à partir de deux loss différentes

Une première loss avec la donnée :

$$L_{data} = \frac{1}{n_{data}} \sum_{i=1}^{n_{data}} (\hat{u}(x_i, y_i, t_i) - u(x_i, y_i, t_i))^2 + (\hat{v}(x_i, y_i, t_i) - v(x_i, y_i, t_i))^2 + (\hat{p}(x_i, y_i, t_i) - p(x_i, y_i, t_i))^2$$

Mais nous savons aussi que notre réseau de neurone doit en théorie satisfaire l'équation de Navier Stokes. On peut donc calculer ces grandeurs :

$$\begin{aligned} R_1(x, y, t) &= \frac{\partial \hat{u}}{\partial t} + \hat{u} \frac{\partial \hat{u}}{\partial x} + \hat{v} \frac{\partial \hat{u}}{\partial y} + \frac{1}{\rho} \frac{\partial \hat{p}}{\partial x} - \nu \left(\frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} \right) \\ R_2(x, y, t) &= \frac{\partial \hat{v}}{\partial t} + \hat{u} \frac{\partial \hat{v}}{\partial x} + \hat{v} \frac{\partial \hat{v}}{\partial y} + \frac{1}{\rho} \frac{\partial \hat{p}}{\partial y} - \nu \left(\frac{\partial^2 \hat{v}}{\partial x^2} + \frac{\partial^2 \hat{v}}{\partial y^2} \right) \\ R_3(x, y, t) &= \frac{\partial \hat{u}}{\partial x} + \frac{\partial \hat{v}}{\partial y} \end{aligned}$$

Ces trois grandeurs que l'on appelle résidus sont très simples à calculer de part l'architecture des réseaux de neurones. En utilisant la règle de la chaîne on peut facilement revenir à toutes les dérivées des résidus et vérifier que le MLP satisfait Navier-Stokes sur n'importe quels points. En utilisant les modules de pytorch, on obtient facilement les valeurs des dérivés.

Finalement, on obtient notre deuxième loss :

$$L_{pde} = \frac{1}{n_{pde}} \sum_{i=1}^{n_{pde}} R_1(x_i, y_i, t_i)^2 + R_2(x_i, y_i, t_i)^2 + R_3(x_i, y_i, t_i)^2$$

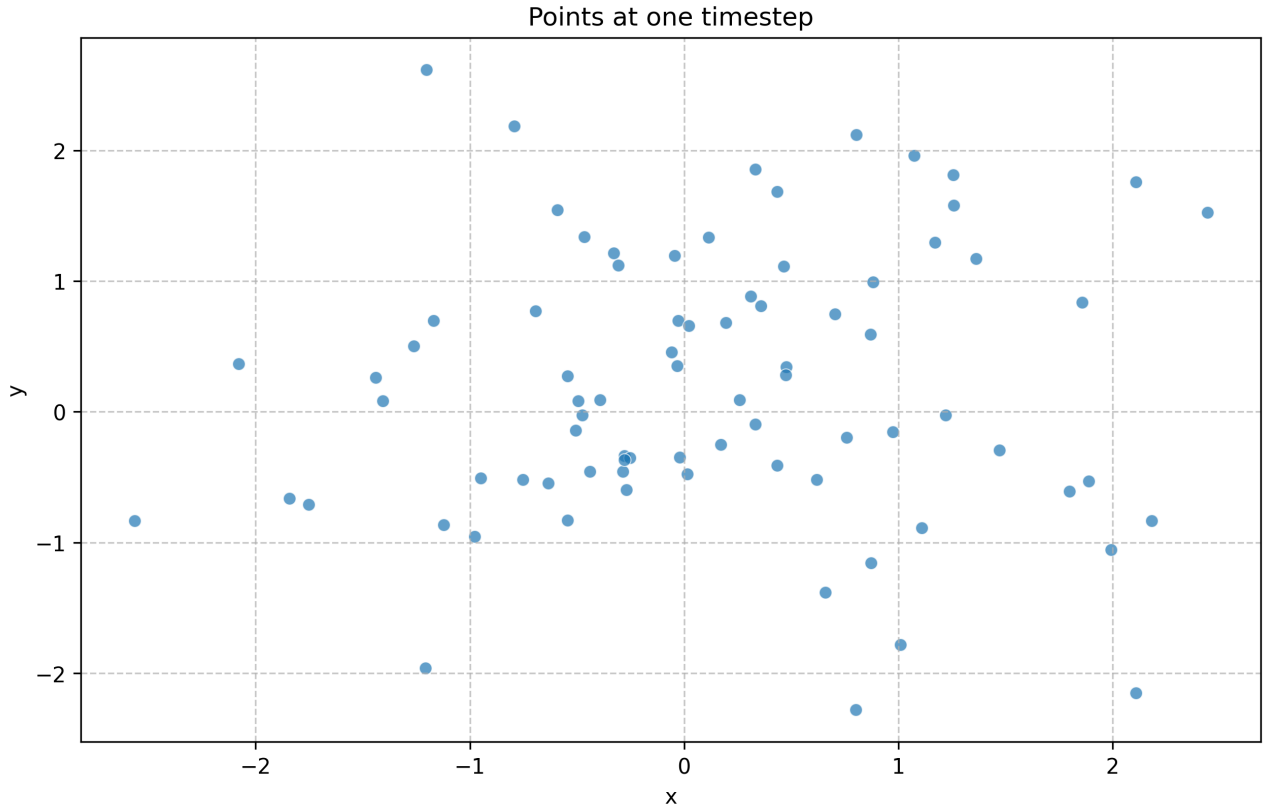
On cherche alors à optimiser la somme pondérée de ces deux loss :

$$L_{tot} = \lambda_{data} L_{data} + \lambda_{pde} L_{pde}$$

où λ_{data} et λ_{pde} sont des hyperparamètres du modèle.

2.2 Présentation du problème

Nous allons chercher à entraîner notre modèle sur un écoulement existant. Nous allons l'entraîner uniquement sur un nombre de points limité :



L'objectif va être de voir si le PINNs permet de faire une bonne interpolation comparé au réseau de neurone classique.

Ce genre de problème a été fait dans ce papier [Xu et al. \[2022\]](#). Dans un premier temps j'avais essayé de juste reproduire les résultats du papier avant de le faire avec les simulations de John Redford.

2.3 Prise en compte des forces inertielles

Dans notre exemple nous allons prendre en compte les forces inertielles. Notre câble va être en mouvement et nous nous plaçons dans le référentiel du cylindre. Notre référentiel n'est plus Galiléen, il faut donc prendre en compte les forces inertielles. Dans ce papier de simulation numérique, on peut voir comment cela est fait [Zhang et al. \[2020\]](#).

L'équation de Navier-Stokes est alors changée de la manière suivante :

$$\tilde{v}_{\tilde{t}} + \tilde{u}\tilde{v}_{\tilde{x}} + \tilde{v}\tilde{v}_{\tilde{y}} = -\frac{1}{\rho}\tilde{p}_{\tilde{y}} + \nu(\tilde{v}_{\tilde{x}\tilde{x}} + \tilde{v}_{\tilde{y}\tilde{y}}) - A_{\tilde{y}}(\tilde{t})$$

où $A(\tilde{t})$ est l'amplitude en y du cylindre.

J'ai eu beaucoup de mal à bien faire tourner les simulations qui utilisent les forces inertielles, mes PINNs fonctionnaient mieux lorsque je ne les mettais pas. Je ne sais pas si cela est dû à une erreur de ma part dans le code ou que l'optimisation était juste trop complexe.

2.4 Utilisation de la normalisation Z

Dans les réseaux de neurones classiques, il est important de normaliser ses données afin que toutes les variables soient du même ordre de grandeur et que le réseau les traite de la même manière.

Il est courant pour chaque grandeur de les normaliser de cette manière :

$$X^* = \frac{X - \bar{X}}{\sigma_X}$$

Ainsi la moyenne de X^* vaut 0 et son écart type vaut 1.

Le problème de cette normalisation dans notre cas est qu'elle va changer les équations que nos variables satisfont. Ce problème est résolu dans ce papier : [Xu et al. \[2024\]](#).

En prenant les mêmes notations que dans ce papier, on adimensionne :

$$v_t + uv_x + vv_y = -p_y + \frac{1}{Re} (v_{yy} + v_{xx}) - \frac{A(t).L}{U_\infty^2}$$

Puis on utilise notre normalisation Z :

$$\frac{\sigma_v}{\sigma_t} v_{t^*}^* + \hat{u} \frac{\sigma_v}{\sigma_x} v_{x^*}^* + \hat{v} \frac{\sigma_v}{\sigma_y} v_{y^*}^* + \frac{\sigma_p}{\sigma_y} p_{y^*}^* - \frac{1}{Re} \left(\frac{\sigma_v}{\sigma_x^2} v_{x^* x^*}^* + \frac{\sigma_v}{\sigma_y^2} v_{y^* y^*}^* \right) + \frac{A(t^*.\sigma_t + \bar{t}).L}{U_\infty^2} = 0$$

2.5 Stratégie de mooving weights

L'équation finale que l'on cherche à optimiser est :

$$L_{tot} = \lambda_{pde} L_{pde} + \lambda_{data} L_{data}$$

Le problème est que le choix des poids est arbitraire et qu'il peut grandement changer la manière dont se fait l'optimisation, voire le résultat final de celle-ci. Dans ce papier : [Goswami et al. \[2022b\]](#) ils utilisent une méthode très simple pour que les poids bougent de façon à aider l'optimisation.

A la fin de chaque epoch, on modifie les poids de cette manière :

On fait une montée de gradient de ce type :

$$\hat{\lambda}_{k+1} = \lambda_k + \eta_k \cdot \nabla \lambda_k$$

Avec η_k le learning rate.

Ensuite on renormalise à 1 la somme des poids de loss :

$$\lambda_{k+1} = \frac{\hat{\lambda}_{k+1}}{\sum \hat{\lambda}_{k+1}}$$

Le gradient de la loss par rapport aux poids est très simple :

$$\frac{\partial \lambda_i}{\partial L} = L_i$$

En pratique j'ai trouvé que pour certains problèmes cette méthode m'aide grandement à ce que le réseau converge plus rapidement.

2.6 Résultats et comparaison avec l'apprentissage supervisé classique

On pourra retrouver tous les hyperparamètres utilisés dans le code dans le dossier result et le fichier hyperparam.json

Apprentissage supervisé

Pour l'apprentissage supervisé classique, on se rend compte que l'on tombe dans un overfitting directement. En effet, il n'y a pas beaucoup de points et le réseau apprend par coeur très facilement tous les points.



FIGURE 2 – Loss pour l'apprentissage non supervisé

Lorsqu'on essaye de plot visuellement les écoulements, on se rend très vite compte que ça ne marche pas.

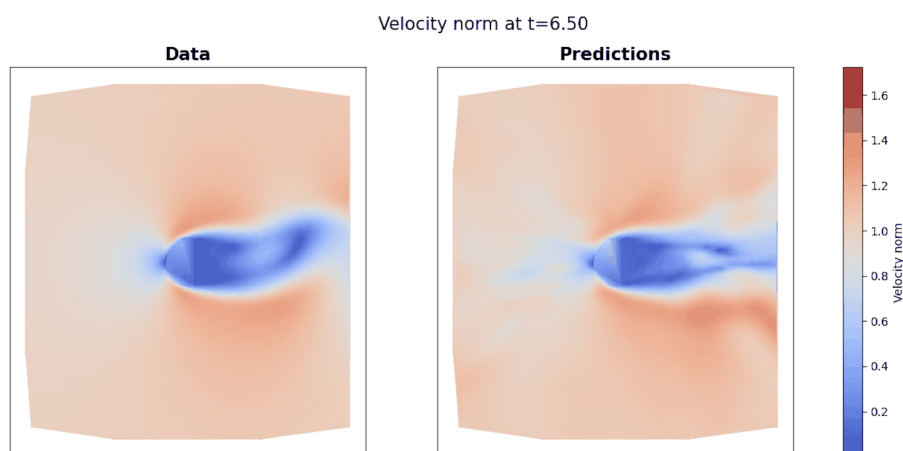


FIGURE 3 – Ecoulement pour l'apprentissage supervisé

Apprentissage avec les PINNs

Avec les PINNs on va aussi avoir de l'overfitting mais le fait d'avoir le terme physique permet au réseau de converger tout de même. (On voit un saut bizarre vers l'époch 250 car j'ai relancé le code avec le modèle un peu entraîné, il y a donc de nouvelles données qui expliquent cette discontinuité). Pour cette simulation j'ai pris un réseau très profond (80 000 paramètres). C'est pour cela que l'on voit un si gros overfitting, cependant la structure donnée par les PINNs permet de continuer de converger et d'atteindre des valeurs très faibles.

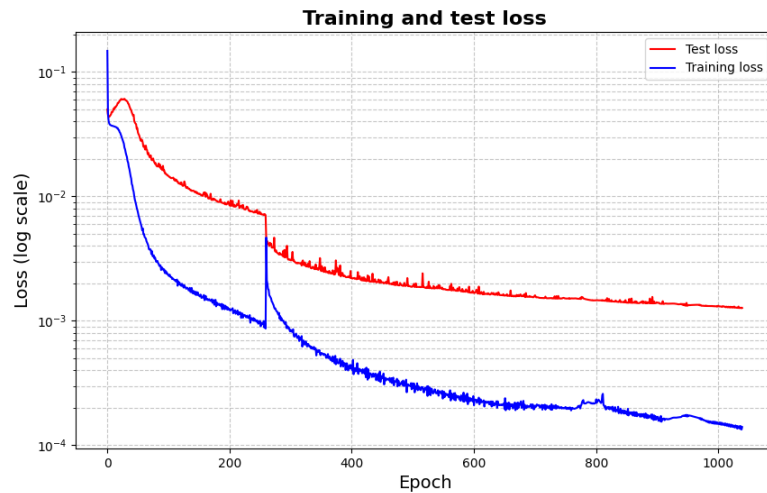


FIGURE 4 – Loss pour l'apprentissage avec les PINNs

On obtient par ailleurs de bons résultats visuels.

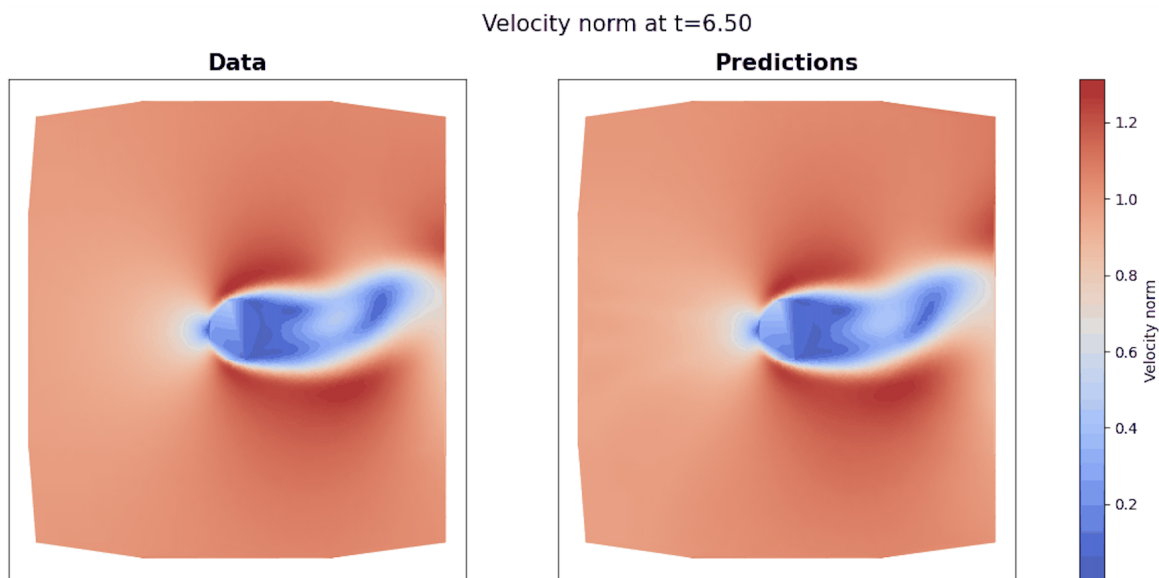


FIGURE 5 – Ecoulement pour l'apprentissage avec les PINNs

En conclusion il semble qu'à l'aide des PINNs on peut bien reconstruire des écoulements que l'on arrive pas à reconstruire avec de l'apprentissage supervisé. La loss liée à l'EDP permet de donner une structure plus physique au réseau et il nous donne de meilleurs résultats.

3 Utilisation des Physics Informed Kolmogorov-Arnold Networks (PIKANs)

Le code utilisé dans cette partie est trouvable sur ce lien : [lien du github](#)

3.1 Principe

Principe du réseau de neurones

Les réseaux de neurones classiques sont basés sur un théorème d'approximation :

Soit $x \in \mathbb{R}^n$
Soit $f : x \mapsto f(x) \in \mathbb{R}$
Alors, il existe $w_i, b_i \in \mathbb{R}^n, a_i \in \mathbb{R}$ tels que
$$f(x) \approx \sum_i a_i \sigma(w_i x + b_i)$$

Avec σ une fonction non linéaire (tanh par exemple)

Ce théorème nous dit qu'un réseau de neurones avec une seule couche mais avec une profondeur arbitrairement grande peut approximer n'importe quelle fonction à valeur réelle.

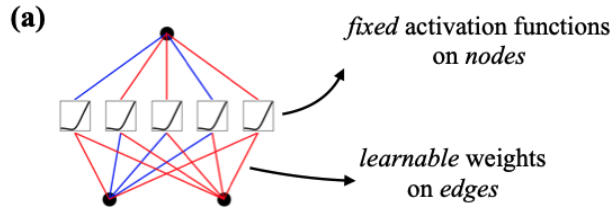


FIGURE 6 – MLP avec une couche

Empiriquement, nous nous sommes rendu compte qu'en mettant plusieurs couches on peut arriver à de meilleurs résultats. C'est de là que vient le deep learning classique.

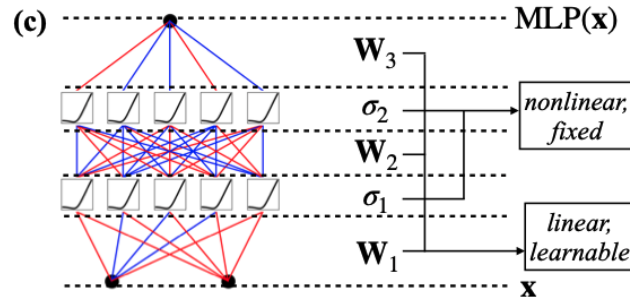


FIGURE 7 – MLP avec plusieurs couches

Un nouveau théorème d'approximation

Le papier [Liu et al. \[2025\]](#) présente les Kolmogorov-Arnold Networks qui sont eux basé sur un nouveau théorème d'approximation :

Soit $x \in \mathbb{R}^n$

Soit $f : x \rightarrow f(x) \in \mathbb{R}$

Alors, il existe $\psi_i, \phi_{i,q}$ des fonctions à valeurs réelles telles que

$$f(x) \approx \sum_q \psi_i \left(\sum_p \phi_{q,p}(x_p) \right)$$

Ce théorème nous dit que n'importe quelle fonction à valeurs réelles peut être décomposée en une somme de fonctions simples appliquées aux composantes de x . Nous n'apprenons plus les poids des MLP mais directement les fonctions d'activation.

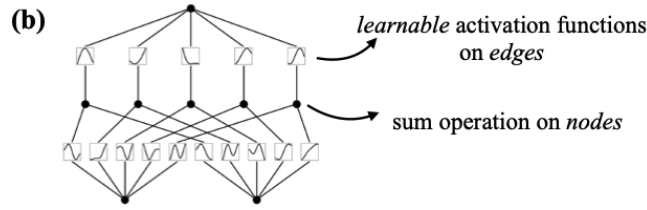


FIGURE 8 – KAN avec une couche

Trouver ces fonctions simples

Afin de trouver ces fonctions simples, nous avons utilisé les B-Splines. Ce type de fonctions permet d'approximer un grand nombre de fonctions avec des paramètres variables. Ce sont ces paramètres que l'on apprend dans le cas des KANs. Ces paramètres sont les points verts de la courbe suivante qui définissent entièrement la fonction approximée.

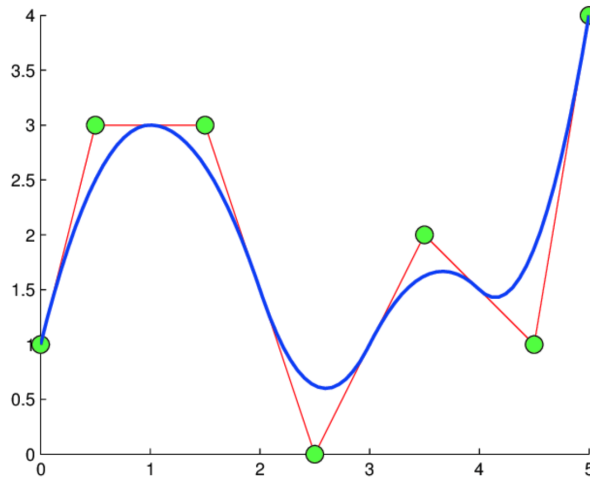


FIGURE 9 – Courbe construite à partir de B-spline

Aujourd'hui il est plus courant d'utiliser les polynômes de Tchebychev afin d'approximer ces fonctions simples, ils semblent donner de meilleurs résultats.

De la même manière que pour le deep learning, on peut s'imaginer mettre plus de couches faites de fonctions simples et espérer un meilleur fonctionnement du réseau. C'est toute l'idée du papier fondateur.

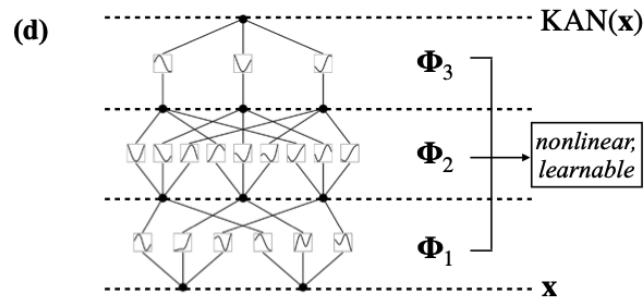


FIGURE 10 – KAN avec plusieurs couches

Philosophie des KANs

L'idée que nos fonctions physiques seraient des sommes de fonctions simples appliqués à chaque composante donne plusieurs avantages :

- Pour un réseau de neurones assez simple on peut alors avoir plus d'interprétabilité, on peut retrouver à partir des fonctions apprises des fonctions simples connues comme le sinus, la fonction carré... On peut donc avoir une formule explicite.

- Philosophiquement, les solutions classiques en physique sont des sommes et des compositions de fonctions simples connues. En partant de ce constat, ce type d'approximation pour nos solutions semblent avoir la bonne structure et devrait pouvoir nous donner de bons résultats.

Les premiers résultats du papier montrent que l'on peut apprendre des fonctions avec beaucoup moins de paramètres. L'entraînement est cependant beaucoup plus long.

PIKANs

Bien sûr, comme pour les PINNs, nos réseaux KANs étant différentiables, on peut utiliser la même stratégie pour ajouter de la physique. C'est ce qui est fait dans ce papier [Toscano et al. \[2024b\]](#)

3.2 Résultats et discussion

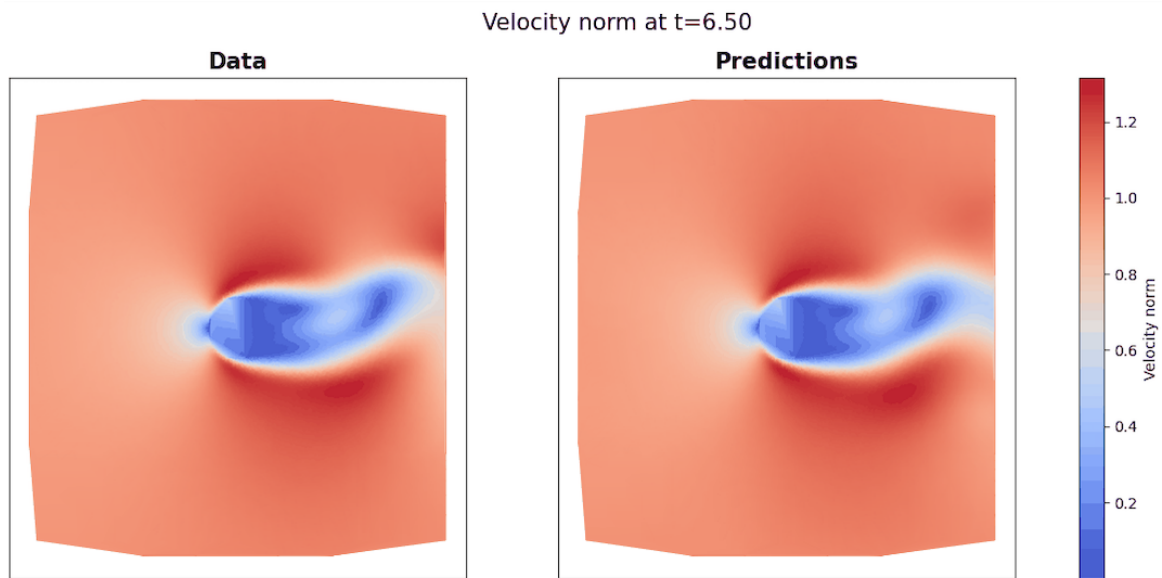
On essaye de reproduire la reconstruction d'écoulement mais à l'aide des PIKANs. L'implémentation des PIKANs provient de ce github : [github pikans](#)

Les pikans sont beaucoup plus longs à converger (on compte un ordre de grandeur supérieur). En revanche on réussit bien à reconstruire l'écoulement :



FIGURE 11 – Loss avec PIKAN

On remarque que visuellement, on reconstruit bien l'écoulement :



Maintenant je n'ai pas plus creusé les PIKANs, j'aurais pu les comparer avec les MLP ou essayer de continuer avec eux mais j'ai préféré m'intéresser à d'autres choses comme je l'explique dans la conclusion.

4 Prédictions d'écoulements avec les PINNs

Le code utilisé dans cette partie est trouvable sur ce lien : [lien du github](#)

4.1 Présentation du problème

Dans la partie de reconstruction d'écoulements, nous avons vu que les PINNs peuvent nous aider à avoir de meilleurs résultats sur des simulations physiques. Nous allons maintenant essayer de les réutiliser afin de prédire des écoulements sur lesquels nous n'avons pas entraîné notre modèle.

On considère un cylindre en mouvement avec une vitesse $s(t) = -y_0\omega_0 \sin(\omega_0 t)$ où y_0 est l'amplitude d'oscillation et ω_0 représente la pulsation de l'écoulement.

Notre objectif va être d'entraîner un PINN avec différentes valeurs de ω_0 et y_0 pour pouvoir prédire d'autres écoulements sur d'autres valeurs.

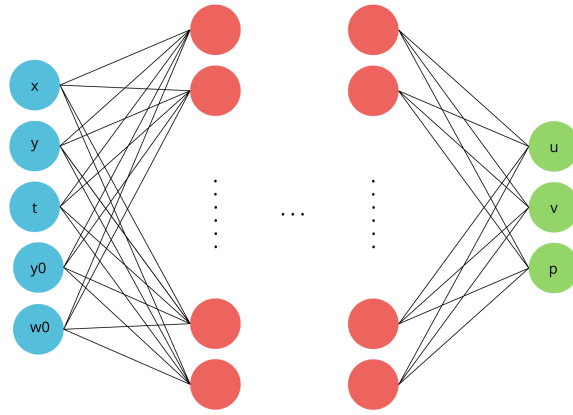


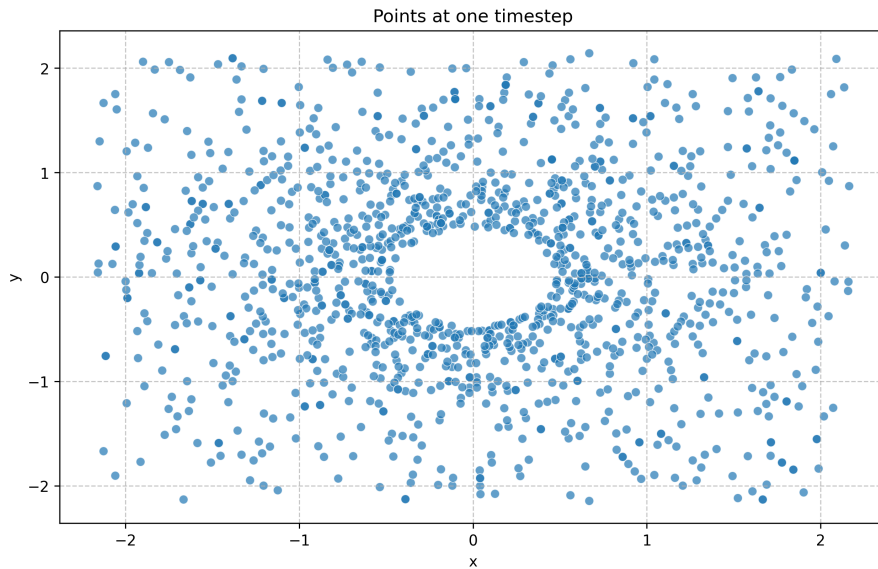
FIGURE 12 – Structure du PINN

L'objectif final va être d'avoir les forces de pression autour du cylindre afin de bien prédire la portance. Pour être plus précis, nous allons ajouter une loss pour les conditions de bord sur le cylindre. La vitesse doit être nulle et la pression doit valoir les valeurs de pression données par les simulations.

Notre Loss totale devient alors :

$$L_{tot} = \lambda_{data}L_{data} + \lambda_{pde}L_{pde} + \lambda_{border}L_{border}$$

Ici notre objectif n'est pas de reconstruire l'écoulement. Nous pouvons donc prendre plus de points et des pas de temps moins espacés.



On entraîne sur 18 simulations différentes : 2 valeurs de ω_0 et 9 valeurs de y_0

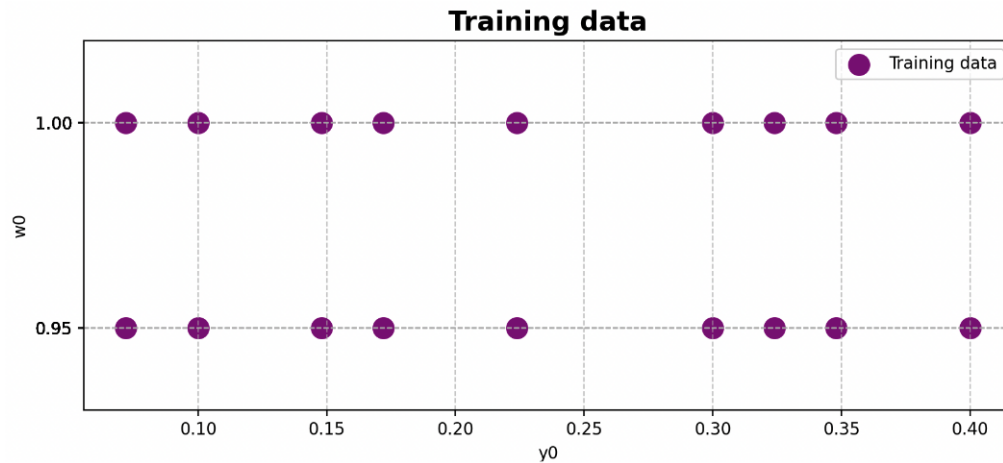
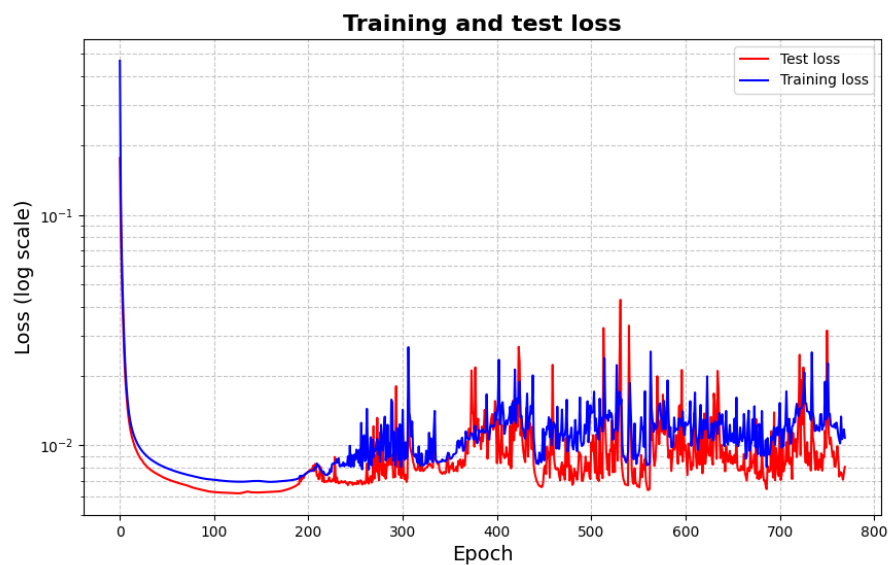


FIGURE 13 – Données utilisées pour l'entraînement

4.2 Résultats

Notre réseau converge bien :



Je n'explique pas ce bruit à la fin, je vais donc utiliser le réseau entraîné vers l'epoch 130.

Prédiction d'un écoulement

On peut chercher à reproduire un écoulement sur lequel on a pas entraîné le modèle.

On prend cette valeur de y_0 et de ω_0 :

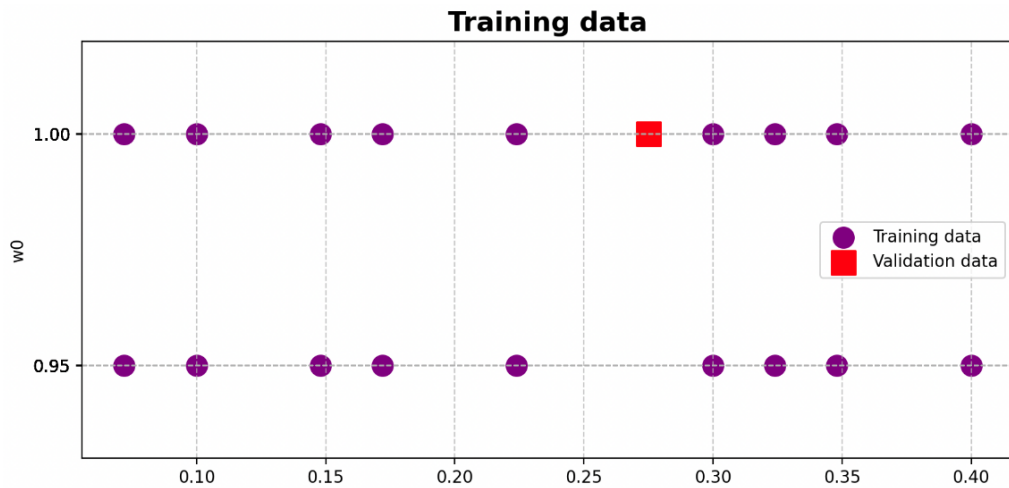
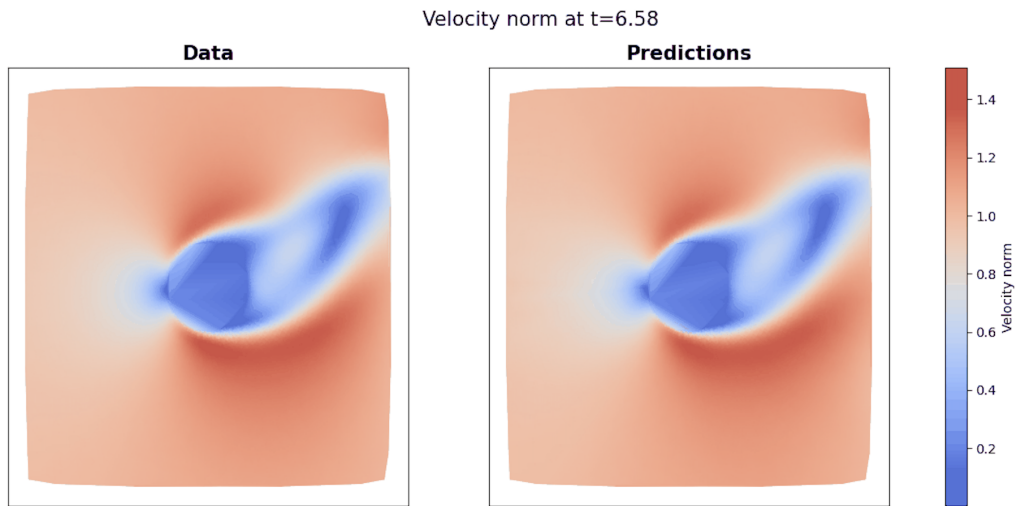


FIGURE 14 – Données utilisées pour la validation

On peut alors visualiser l'écoulement et l'on arrive bien à le prédire sur des données de validation :



Cependant ce qui nous intéresse, ce sont les forces de portance sur le cylindre.

La force de portance est donnée par cette formule :

$$F_l = - \int_0^{2\pi} P(\theta) r \sin(\theta) d\theta$$

Par une intégration numérique on peut calculer cette valeur puis on peut sortir le coefficient de portance avec cette formule :

$$C_l = \frac{F_l}{\frac{1}{2} \rho V^2 S}$$

Avec ρ la masse volumique de l'air, V la vitesse d'entrée et S la surface sur laquelle s'exerce les forces.

Ici j'ai pris $S = 2\pi R$ Avec R le rayon du cylindre. On considère donc une longueur unitaire de câble.

On cherche à valider notre modèle sur des valeurs où l'on ne l'a pas entraîné :

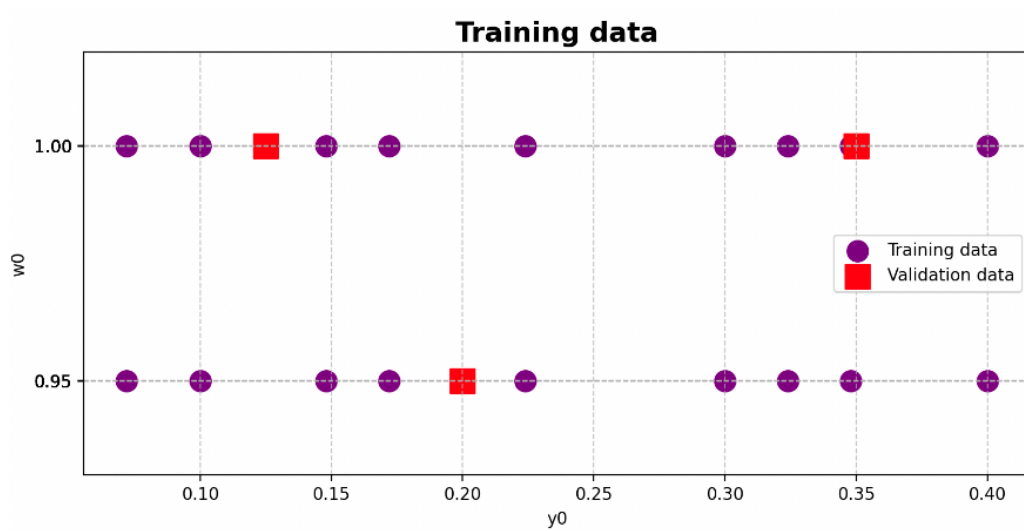


FIGURE 15 – Données utilisées pour la validation

On trouve alors de très bons résultats :

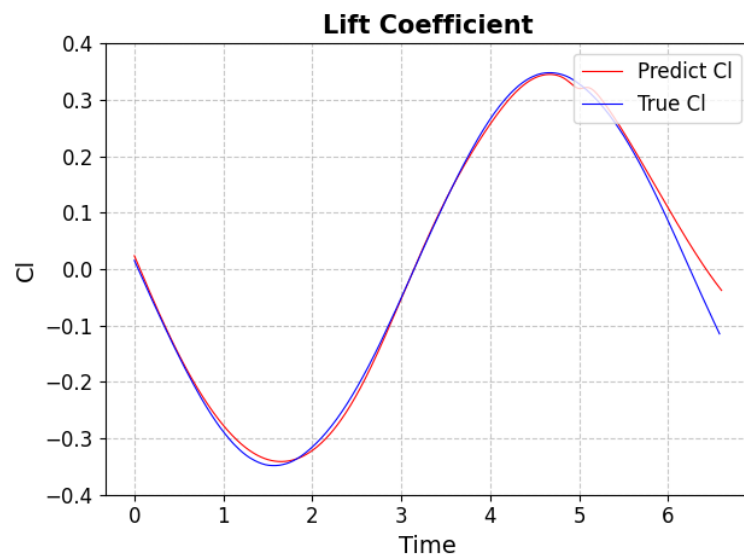


FIGURE 16 – $y_0 = 0.125$, $w_0 = 1.0$

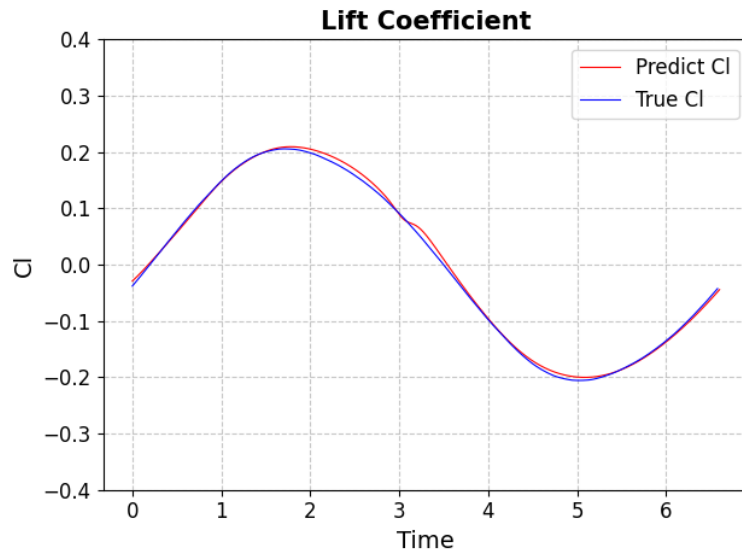


FIGURE 17 – $y_0 = 0.2$, $w_0=0.95$

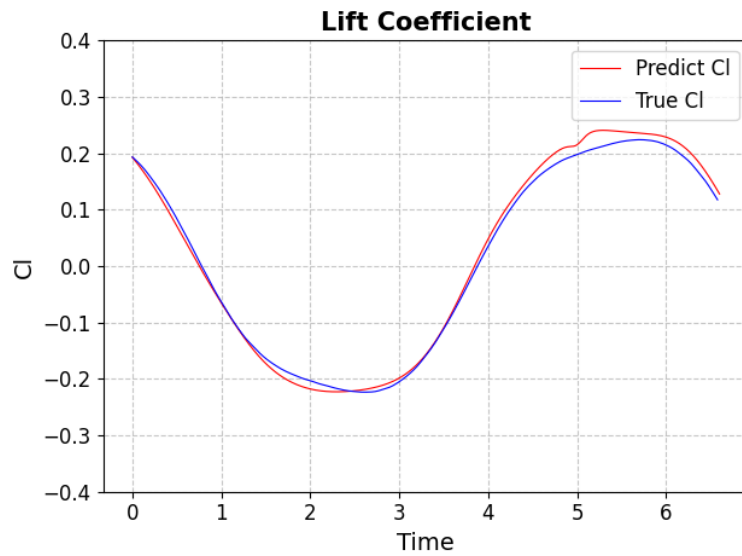


FIGURE 18 – $y_0 = 0.35$, $w_0=1.0$

En conclusion avec ce réseau de neurone entraîné, nous pouvons prédire des écoulements dans l'intervalle de l'entraînement ainsi que prédire les forces de portance qui s'exercent sur le câble.

Je n'ai pas remis les résultats ici mais les prédictions fonctionnent très bien aussi sans utilisé les PINNs, uniquement avec des MLP. Comme l'on a des points beaucoup moins espacé en temps et en espace, notre réseau de nerone est capable d'apprendre sans les PINNs.

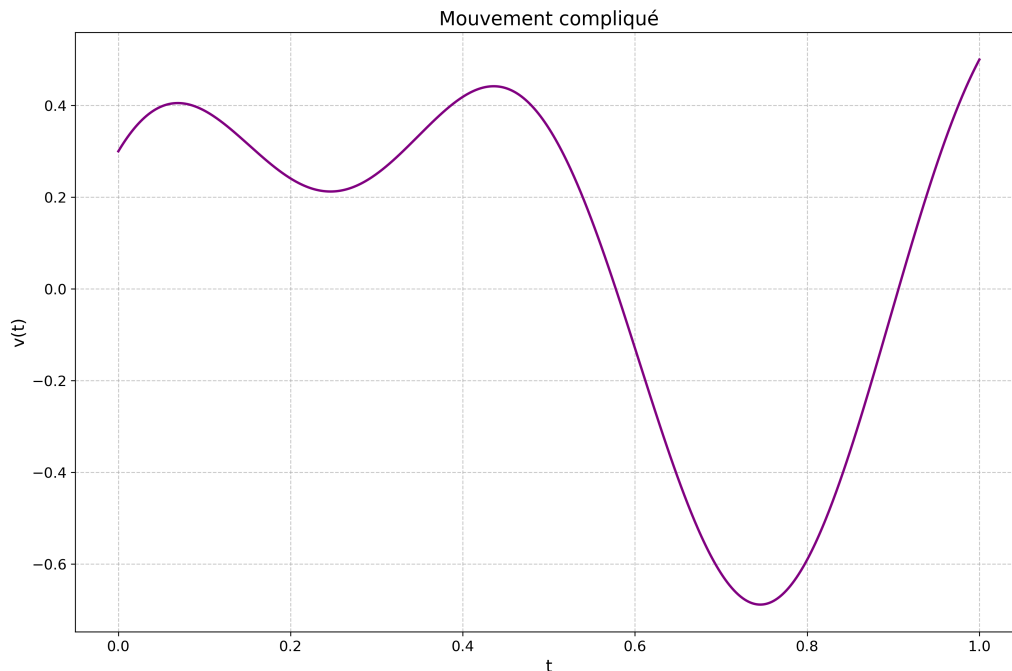
5 Deep Operator Network (Deep O Net) pour la prédiction d'écoulements

Le code utilisé dans cette partie est trouvable sur ce lien : [lien du github](#)

5.1 Principes

Limite de notre implémentation

Dans la partie prédiction d'écoulements, on encode notre mouvement du cylindre à l'aide des valeurs de ω_0 et de y_0 . Comme tous nos mouvements de cylindre sont sinusoidaux il est très simple de les encoder avec uniquement deux valeurs. Cependant, pour des mouvements plus complexes cela n'est plus possible. Il faut donc un autre type de modèle.

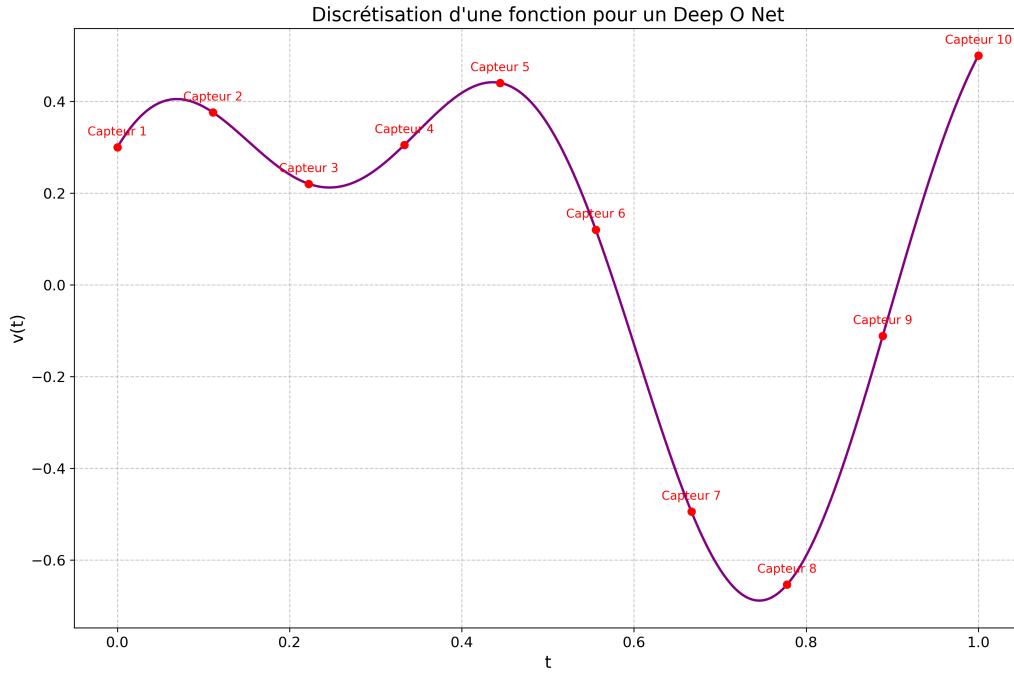


Deep Operateur Network

Dans le domaine de la prévision de grandeurs physiques, les PINNs sont très peu utilisés, on utilise bien plus les Neural Operators. Ils permettent d'apprendre avec des fonctions d'entrée de forme arbitraire. Le papier fondateur est [Lu et al. \[2021a\]](#). Bien sûr, on retrouve son pendant informé par la physique dans ce papier [Goswami et al. \[2022a\]](#). Je n'ai implémenté que le Deep O Net classique.

Apprendre un mouvement arbitraire

Afin d'apprendre un mouvement arbitraire, on va discrétiser notre fonction d'entrée sur des temps fixes, on les appelle des capteurs :



La discrétisation du graphique sur l'image est assez grossière mais en prenant encore plus de capteurs, on peut avoir une bonne représentation de la fonction d'entrée.

Un nouveau théorème d'approximation

Ici notre problème est un peu différent du problème de reconstruction de fluide. On ne cherche plus à mapper de (x, y, t) dans $(u(x, y, t), v(x, y, t), p(x, y, t))$ donc d'un espace de dimension 3 vers un espace de dimension 3. Maintenant notre objectif est de mapper de (s) la fonction de vitesse du cylindre vers (u, v, p) vu cette fois comme des fonctions et non plus évalués en des points. On mappe d'un espace fonctionnel vers un espace fonctionnel. C'est ce qu'on appelle un opérateur.

De la même manière que l'on a un théorème d'approximation universel des fonctions, on va avoir un théorème d'approximation universel des opérateurs :

Soit un opérateur G qui, à partir de la vitesse du cylindre s , donne la fonction du champ de vitesse u en (x, y, t) .

Alors, il existe des coefficients $c_i^k, \xi_{ij}^k, \theta_i^k, \omega_k, \zeta_k$ tels que :

$$u \approx G(s)(X = (x, y, t)) = \sum_k \sum_i c_i^k \sigma \left(\sum_j \xi_{ij}^k s(t_j) + \theta_i^k \right) \sigma(\omega_k \cdot X + \zeta_k),$$

où σ est une fonction non linéaire (par exemple, $\sigma(x) = \tanh(x)$).

Ce théorème ressemble en fait grandement au théorème d'approximation universel des fonctions. Il dit que l'on considère une couche arbitrairement grande pour gérer les données des capteurs (la vitesse du câble), cette couche est appelée branch net. On a une couche de la même taille pour gérer les coordonnées appelée trunk net. On fait ensuite le produit scalaire de ces deux couches pour avoir notre grandeur de sortie (ici le champ de vitesse).

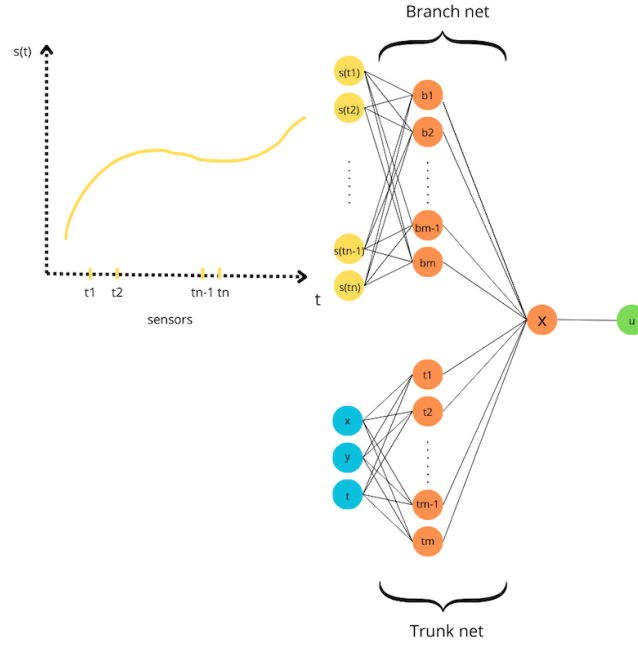


FIGURE 19 – Architecture

De la même manière que pour le Deep Learning, on peut augmenter le nombre de couches pour avoir de meilleurs résultats.

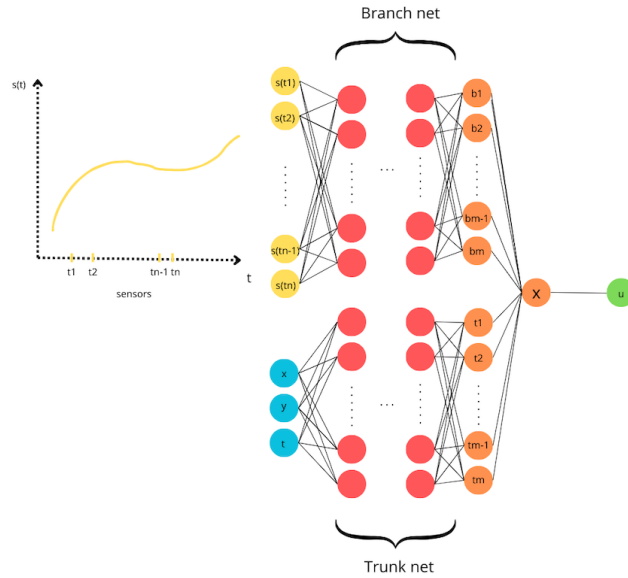


FIGURE 20 – Architecture d'un Deep O Net

Dans cette implémentation on cherche uniquement à reproduire les résultats de la partie prédictions d'écoulements avec cette architecture et la prise en compte de la vitesse du cylindre à l'aide des capteurs (on a une information moins bien structurée).

5.2 Résultats et discussion

De la même manière que pour les PIKANs, j'ai implémenté l'Opérateur et une fois qu'il a fonctionné je n'ai pas plus approfondi. Il y a quand même une bonne base sur laquelle s'appuyer. On peut reproduire les résultats de prédiction d'écoulement avec ce type de réseau. On retrouvera les courbes et les résultats sur le github.

On peut maintenant essayer de le faire fonctionner pour des écoulements plus complexes que les PINNs ne peuvent pas gérer.

6 Utilisation des Graph Neural Network (GNN)

Lors de la conférence DTE-AICOMAS à laquelle nous avons pu participer, j'ai assisté à la présentation de Théodore Michel un doctorant des Mines de Paris dans le laboratoire du CEMEF. Le laboratoire à une problématique très proche de la nôtre car il cherche à faire beaucoup plus rapidement certaines simulations de mécanique des fluides afin d'obtenir la portance et mettre tout ça dans un modèle. On trouvera ici leur papier sur le sujet : [Garnier et al. \[2024\]](#). Je pense que pour notre sujet, cette approche est la bonne car nous pouvons garder cette structure de maillage inhérente aux simulations numériques. On sait déjà que c'est possible de cette manière alors qu'il n'y a que très peu (voir pas) d'exemples avec les PINNs classiques. D'autant plus qu'en discutant avec Théodore, le CEMEF a une grosse compétence en simulation de mécanique des fluides et ils peuvent transférer cette compétence sur les GNN (comme le raffinement de maillage par exemple). On peut s'imaginer faire la même chose avec John et Mews Lab.

On pourra trouver le code sur les gnn ici : [github GNN](#). Je n'ai utilisé aucun module pour faire des graphs mais il est peut être judicieux d'utiliser le module pytorch Geometric : [doc](#)

6.1 Principe

Le papier fondateur pour faire des prévisions d'écoulements avec des GNN est celui de deep mind : [Pfaff et al. \[2021\]](#). Je vais essayer d'expliquer le fonctionnement ici mais il ne faut pas hésiter à lire ce papier au moindre doute.

Avec les GNN, on garde la structure maillée de notre simulation et la formulation Eulerienne de notre problème :

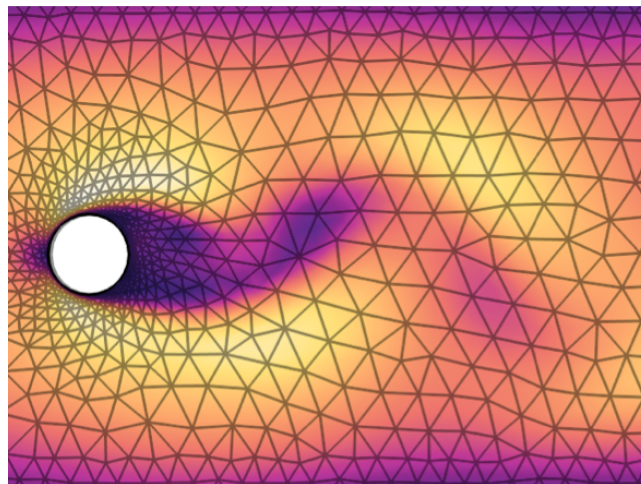


FIGURE 21 – Maillage d'une simulation numérique.

Les noeuds vont avoir les valeurs de vitesse et pression et les arêtes vont avoir les valeurs de distance entre chaque noeud voisins.

Notations

- Les nœuds sont notés V .
- Les valeurs des arêtes sont notées E .
- Chaque nœud i a un nombre $N_n \in \mathbb{N}$ de voisins.
- Le nombre total de nœuds est noté $N_v \in \mathbb{N}$.

Représentation des noeuds

Chaque nœud possède un indice dans V et est représenté par une matrice $V \in \mathbb{R}^{N_v \times 3}$, où :

$V_{i,0}$: vitesse en x du point i ,

$V_{i,1}$: vitesse en y du point i ,

$V_{i,2}$: champ de pression au point i .

Représentation des Arêtes

Pour coder les valeurs des arêtes, nous utilisons une matrice $E \in \mathbb{R}^{N_v \times N_n \times 3}$, où $E_{i,j}$ représente la valeur de l'arête entre le nœud i et son j -ème voisin. On a :

$E_{i,j,0}$: la distance sur x entre le point i et j

$E_{i,j,1}$: la distance sur y entre le point i et j

$E_{i,j,2}$: la norme de la distance entre i et j

Représentation des Voisins

Pour connaître les voisins, nous avons besoin d'une matrice supplémentaire $E_n \in \mathbb{R}^{N_v \times N_n}$, où $E_{n,i,j}$ donne l'indice du j -ème voisin du i -ème point.

Principe de fonctionnement

Le principe de fonctionnement va être qu'à un instant t , on a notre graph $G_t = (V, E, E_n)$. L'objectif va être d'obtenir le graph sur le pas de temps suivant G_{t+1} .

Pour cela, on utilise notre GNN comme un opérateur qui va approximer notre dérivé temporelle :

$$GNN(G_t) = \dot{V}_t$$

Ensuite avec une intégration numérique on obtient le pas suivant :

$$V_{t+1} = GNN(G_t)\Delta_t + V_t$$

Le GNN fonctionne en 3 phases : une phase d'encoding où on met nos matrices dans un espace plus gros, une phase de processing où les noeuds prennent des informations des voisins, on appelle ça le message passing. Puis une phase de decoding où l'on récupère $\dot{V}_t$

Phase d'encoding

On encode V et E dans un espace plus grand de taille N_e . Dans le papier de deep mind, $N_e = 128$. On encode cela simplement avec un multi layer perceptron classique (MLP). On obtient alors :

$$\tilde{V} \in \mathbb{R}^{N_v \times N_e}$$

$$\tilde{E} \in \mathbb{R}^{N_v \times N_n \times N_e}$$

Phase de processing

On va maintenant faire le processing. On va mettre l'information des voisins dans chaque noeud. On peut répéter cette étape plusieurs fois. Plus on va répéter cette étape, plus on va avoir sur chaque noeud l'information de noeuds plus éloignés.

Ce processus se fait en trois étapes, dans la première on réunit l'information des noeuds voisins et de leurs arêtes. Puis pour chaque noeud on somme ces informations ce qui fait un message. Enfin on actualise la valeur des noeuds avec ce message et l'ancienne valeur de noeud. Le tout avec des MLP et des branches résiduelles

Processing :

$$\begin{aligned}\tilde{E}'_{i,j} &= MLP(\tilde{V}_i, \tilde{V}_k, \tilde{E}_{i,j}), \text{ où } k \text{ est l'indice du jème voisin de } i \\ M_i &= \sum_j \tilde{E}'_{i,j} \\ \tilde{V}'_i &= \tilde{V}_i + MLP(\tilde{V}_i, M_i)\end{aligned}$$

Phase de decoding

On décode pour obtenir $\dot{V} = MLP(\tilde{V}', \tilde{E}') \in \mathbb{R}^{N_v \times 3}$

Astuces pour améliorer le modèle

Le processus est itératif (il faut G_t pour avoir G_{t+1}) les erreurs s'accumulent donc au fur et à mesure que l'on avance dans le temps. Une méthode d'ajout de bruit pendant l'entraînement a été développée afin de rendre le modèle plus robuste.

Il est aussi possible de faire du raffinement de maillage au fur et à mesure de la simulation. Je n'ai malheureusement pas eu le temps d'explorer cette voie.

Dans le papier de l'équipe du CEMEF [Garnier et al. \[2024\]](#), ils utilisent des mécanismes d'attention pour aider leur modèle. Je n'ai pas pu creuser cette possibilité. Ce papier [Veličković et al. \[2018\]](#) est le papier présentant cette méthode.

J'ai aussi entendu parler d'ajouter de la physique à ces GNNs, je crois que le CEMEF devrait bientôt sortir un papier là dessus avec des GNNs/PINNS.

On peut terminer nos MLP par des layer norms pour éviter l'explosion ou la disparition des gradients.

Il est possible d'encoder avec un principe de "one hot encoding" le fait qu'un noeud vienne d'une condition initiale, qu'il soit sur le bord ou qu'il soit dans le domaine.

6.2 Résultats et discussion

Malheureusement j'ai découvert les GNN très tard. J'ai commencé à les implémenter, le réseau de neurone converge mais lorsque j'essaye de prédire l'écoulement, au bout de quelques itérations ça diverge.

J'ai quand même mis mon code sur le github. Je pense qu'avec peu de modifications on peut vite arriver à de bons résultats.

7 Conclusion

7.1 Bilan

En conclusion, je pense que ce stage a permis d'avoir une bonne vue d'ensemble des différentes manières d'accélérer les temps de calcul des simulations de mécanique des fluides avec le machine learning. Nous avons vu l'utilisation des MLP classiques dans les PINNs avec différentes stratégies (comme la Z normalisation...). Ensuite nous avons étudié différentes architectures de réseaux de neurones comme les PIKANs, neural Operator et GNN.

Selon moi les résultats obtenus avec les PINNs classiques pour prédire des écoulements sont encourageants et avec l'évolution très rapide du domaine je pense qu'on pourra très vite avoir facilement de meilleurs résultats.

7.2 Discussion et perspectives

Je vais ici reprendre chaque partie et parler des choses qui sont pour moi importantes à approfondir.

Choix des points de collocations

Le choix des points de collocations peut être un point crucial pour entraîner les PINNs. J'ai essayé différentes choses comme prendre plus de points autour du câble (où on a envie d'être le plus précis). Ou encore d'utiliser un tirage Latin Hypercube Sampling au lieu d'un tirage aléatoire. Ces méthodes n'ont pas forcément augmenté énormément la précision des modèles et prenaient juste plus de temps à tourner. C'est pour cela que maintenant je prends mes points de collocations de manière aléatoire.

Cependant ces points peuvent avoir un gros impact et une étude plus poussée est nécessaire.

PIKANs

Pour les PIKANs, j'ai compris comment ils fonctionnent, puis je les ai implémenté et je n'ai pas plus approfondi. Je pense que la technologie est nouvelle et que ce n'est pas si simple de dire si c'est mieux ou pas qu'un MLP classique. Cet article : [Toscano et al. \[2024a\]](#) du père des PINNs et DeepONet montre que les résultats sont pour l'instant assez mitigés. Il y a des fois où ça fonctionne mieux et des fois où ça fonctionne moins bien. Dans la partie "Advice on hyperparameter tuning" du README du GitHub de l'article fondateur des KANs : [pykan](#), l'auteur dit qu'il est assez dur de transférer son intuition du tuning de MLP au tuning de KANs. Le fait qu'il faille se reconstruire toute une intuition ainsi que le fait que la technologie est nouvelle, sans pour l'instant de réelle plus value sur les MLP classiques fait que je ne suis pas plus partie dans cette voie. Cependant l'architecture reste très intéressante, il faudra suivre la recherche dans le futur.

De plus, j'ai utilisé les B-spline dans mon implémentation des PIKANs mais aujourd'hui les gens utilisent plus les polynômes de Tchebychev.

Utilisation de Deepxde

Une bibliothèque classique pour faire des PINNs est DeepXde : [Deepxde](#). Ce framework est sorti avec le premier papier sur les PINNs : [Lu et al. \[2021b\]](#). J'ai choisi de ne pas l'utiliser car je trouvais très formateur le fait de coder soit même les PINNs autant pour bien comprendre la technologie que pour apprendre pytorch. De plus j'avais essayé d'utiliser un petit peu deepxde et dès que l'on veut faire des choses qui ne sont pas incluses nativement dans le logiciel ça devient très compliqué.

Cependant mon choix n'est peut-être pas le meilleur car Deepxde va être plus robuste et peut-être plus lisible que mon code. Il serait très intéressant de voir si en utilisant deepxde nous pouvons obtenir de meilleurs résultats (en précision et en vitesse de l'optimisation).

Prédictions d'écoulements

Il y a beaucoup de points à étudier plus en profondeur sur la prédiction d'écoulements.

Le fait qu'on ait du mal à bien prendre en compte les forces inertielles en est un premier. Il serait peut-être possible d'utiliser des techniques comme dans un premier temps n'optimiser que la loss sur les données et au fur et à mesure augmenter la loss sur l'équation différentielle.

Ensuite on peut bien sûr essayer de prendre plus de données (notamment en fréquence où on en a essayé que deux)

Deep Neural Operator

Enfin je pense que pour avoir des mouvements de câbles plus complexes nous allons être obligés d'utiliser les Opérateurs. J'ai ici très peu approfondi, un peu à la manière des PIKANs mais c'est l'évolution naturelle de ce travail. La forme des neural operators se prête bien à notre sujet.

GNN

Je n'ai travaillé qu'une semaine sur les GNN mais j'ai l'impression que c'est vraiment la bonne architecture pour notre problème. On a vu des cas plus complexes similaires au nôtre qui fonctionnent. Il faut voir le GNN comme un simulateur, une fois entraîné avec des conditions initiales et des conditions de bord il peut nous sortir l'écoulement et les forces de pression sur le câble. Selon moi c'est la piste à poursuivre le plus.

Encore une fois je n'ai pas implémenté mon GNN avec le module geometric de pytorch, j'ai préféré l'implémenter moi même pour être sûr de bien tout comprendre. Cependant je pense qu'ici le module de pytorch fonctionne réellement mieux que le code que j'ai implémenté, il doit contenir plein d'astuces d'optimisation.

Références

- Paul Garnier, Jonathan Viquerat, and Elie Hachem. Multi-grid graph neural networks with self-attention for computational mechanics, 2024. URL <https://arxiv.org/abs/2409.11899>.
- Somdatta Goswami, Aniruddha Bora, Yue Yu, and George Em Karniadakis. Physics-informed deep neural operator networks, 2022a. URL <https://arxiv.org/abs/2207.05748>.
- Somdatta Goswami, Aniruddha Bora, Yue Yu, and George Em Karniadakis. Physics-informed deep neural operator networks, 2022b. URL <https://arxiv.org/abs/2207.05748>.
- Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. Kan : Kolmogorov-arnold networks, 2025. URL <https://arxiv.org/abs/2404.19756>.
- Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3) :218–229, March 2021a. ISSN 2522-5839. doi : 10.1038/s42256-021-00302-5. URL <https://arxiv.org/abs/1910.03193>.
- Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. Deepxde : A deep learning library for solving differential equations. *SIAM Review*, 63(1) :208–228, January 2021b. ISSN 1095-7200. doi : 10.1137/19m1274067. URL <https://arxiv.org/abs/1907.04502>.
- Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks, 2021. URL <https://arxiv.org/abs/2010.03409>.
- Juan Diego Toscano, Vivek Oommen, Alan John Varghese, Zongren Zou, Nazanin Ahmadi Daryakenari, Chenxi Wu, and George Em Karniadakis. From pinns to pikans : Recent advances in physics-informed machine learning, 2024a. URL <https://arxiv.org/abs/2410.13228>.
- Juan Diego Toscano, Vivek Oommen, Alan John Varghese, Zongren Zou, Nazanin Ahmadi Daryakenari, Chenxi Wu, and George Em Karniadakis. From pinns to pikans : Recent advances in physics-informed machine learning, 2024b. URL <https://arxiv.org/abs/2410.13228>.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks, 2018. URL <https://arxiv.org/abs/1710.10903>.
- Shengfeng Xu, Zhenxu Sun, Renfang Huang, Guo Dilong, Guowei Yang, and Shengjun Ju. A practical approach to flow field reconstruction with sparse or incomplete data through physics informed neural network. *Acta Mechanica Sinica*, 39, 11 2022. doi : 10.1007/s10409-022-22302-x.
- Shengfeng Xu, Chang Yan, Zhenxu Sun, Renfang Huang, Dilong Guo, and Guowei Yang. On the preprocessing of physics-informed neural networks : How to better utilize data in fluid mechanics, 2024. URL <https://arxiv.org/abs/2403.19923>.
- Lei Zhang, J.A. Redford, Fikri Hafid, Jean-Michel Ghidaglia, and Maxime Gueguin. Viv modelled using simplified cable dynamics coupled to sub-critical cylinder flow simulations in a moving reference frame. *European Journal of Mechanics - B/Fluids*, 85, 09 2020. doi : 10.1016/j.euromechflu.2020.09.013.

A Utilisation d'Onyxia pour accéder à des GPU

Onyxia est une plateforme créée par le service public pour les universitaires. Avec une adresse mail étudiante on peut avoir accès à un GPU à distance. Ce système fonctionne super bien grâce à github, on peut mettre notre code sur un VScode en ligne d'Onyxia, lancer le code puis faire un pull pour récupérer les résultats sur github.

Utilisation

- Aller sur ce site : [site d'onyxia](#)
- Cliquer sur 'notre instance' et se créer un compte.
- Mettre les données sur le site : aller dans mes fichiers, téléverser un fichier et charger les données ici.
- Lancer un environnement python avec vscode sur le gpu : aller dans catalogue de service, choisir vscode-pytorch-gpu (si on utilise pytorch), dans le réglage git mettre le https du repo github. Dans Init, personal init, copier le lien raw github du fichier onyxia-init.sh (pour faire tous les pip install automatiquement). Mettre un nom et lancer l'instance.
- On se retrouve alors sur une instance VS code. Il faut télécharger les données que l'on a mis sur onyxia. Ouvrir un terminal dans VS-code, écrire : `'mc cp -r s3/[monnom]/[nomfichier.csv] .'`
- Lancer le code normalement
- Faire un git add, git commit, git push une fois la simulation terminée.

B Apprendre le ML avec internet

Au début de mon stage, je n'y connaissais presque rien en machine learning. J'avais une petite culture et je connaissais bien python mais je n'avais jamais utilisé pytorch. Il y a un grand nombre de contenus de qualité sur internet pour se former gratuitement sur des sujets de ML. Je vais ici recenser ceux que j'ai trouvés les plus utiles dans chaque thématique :

Base en Machine Learning

Les vidéos youtube de Andrej Karpathy, ancien chef du département IA chez Tesla et cofondateur de Open AI. Il a fait une série de vidéos appelées from zero to hero qui présente super bien le machine learning. Il recode un pytorch très basique qui permet de bien comprendre le mécanisme de descente de gradient et comment fonctionne la rétropropagation.

- Playlist sur le machine learning : [From zero to hero](#)
- Une bonne vidéo où il explique la batch normalization (très proche de la layer normalization) : [Batch Normalization](#)
- Sa vidéo où il recode GPT 2 : [GPT 2](#), je trouve qu'il y a beaucoup de choses à apprendre sur cette vidéo, que ce soit pour voir comment il gère pytorch ou encore quand il explique les branches résiduelles.

GNN

- Cet article explique bien comment fonctionnent les gnn : [article medium](#) autrement j'ai surtout lu les papiers.

Mécanisme d'attention et LLM

J'ai voulu apprendre là dessus non seulement parce ce que j'en entends parler partout mais aussi parce que dans des architectures comme les Neural Operators c'est une potentielle future amélioration (cf la conclusion de ce papier : [Lu et al. \[2021a\]](#))

Le mécanisme d'attention étant extrêmement lié aux LLM, je me suis intéressé à ce domaine. Avant de voir l'architecture des transformers, j'ai donc étudié ses ancêtres le recurrent neural network (RNN), le long short term memory (LSTM) et le Sequence to Sequence (Seq2Seq).

- Article sur les RNN (encore de Andrej Karpathy) : [article sur les RNN](#)
- Article sur les transformers : [article sur les transformers](#)
- Une très bonne vidéo pour avoir une intuition du mécanisme d'attention : [vidéo mécanisme d'attention](#)
- Bien sûr toujours la vidéo de Andrej sur GPT 2 : [vidéo GPT 2](#)

Apprendre pytorch

Pour apprendre pytorch j'ai regardé plusieurs petits tutos sans en avoir un qui ait été parfait, j'ai lu la documentation et demandé à chat GPT quand je ne savais pas. J'ai surtout eu l'impression de progresser avec des vidéos où des gens codent en pytorch (comme celle d'Andrej sur GPT 2).